CGC UNIVERSITY, MOHALI

Department of Computer Science & Engineering

PPF WEEK 9 (MINI PROJECT)

Student Record Search System Using Trees (BST)

Data Structures and Algorithms (DSA) - Project Report

Submitted by	Arnav
Roll Number	2556433
Programme	B.Tech CSE
PPF Activity	Data Structures Mini Project - Student Record Search System Using Trees
Assessment Method	Mini Project (20 Marks)
Mode	Individual
Submission	GitHub Repository
Duration	1 Week
Language / IDE	C (GCC) / Visual Studio Code

1. Problem Statement

The aim of this project is to build a Student Record Search System using a Binary Search Tree (BST) implemented in the C programming language. Each record contains a unique enrolment ID (the BST key), the student name and the programme/department. The system supports insertion, searching, deletion and sorted display of records. It demonstrates the BST ordering property, tree traversals, duplicate-key handling, dynamic memory allocation with pointers, and the effect of tree shape on efficiency, and explains average-case and worst-case time complexity.

2. Requirement Coverage (Tasks vs Implementation)

Task	Requirement	How it is implemented
9	Student record structure	`struct Student` with `enrol_id` (key), `name`, `dept`; `struct Node` with `data`, `left`, `right` pointers.
10	Insertion, reject duplicates	`insert()` first searches the key; a duplicate prints a clear message and is rejected (returns 0).
11	Search by enrolment ID	`search()` - iterative; returns the node pointer, or NULL when not found.
12	Deletion: leaf, one child, two children	`delete_node()` handles all 3 cases and frees the removed node; the two-child case uses the inorder successor.
13	Inorder, preorder, postorder	Menu options 5, 6, 7; inorder gives records sorted by ID.
14	At least 10 records, menu-driven	Option 2 loads 10 sample records; menu has 10 options plus Exit.
15	Balanced vs sorted insertion comparison	Option 9 counts comparisons and height for both orders (Section 6).
16	Why O(n) worst case; AVL	Section 7.

3. Data Structure Design

A BST is a binary tree in which, for every node, all keys in the left subtree are smaller and all keys in the right subtree are larger than the node's key. The enrolment ID is the key, so inorder traversal (left, node, right) always visits records in ascending ID order. In C the tree is built from heap-allocated nodes linked with pointers; `malloc()` creates a node on insertion and `free()` releases it on deletion and at program exit, so the program leaves no memory leaks.

Structure	Fields	Purpose
`Student`	enrol_id (int, key), name (char[32]), dept (char[16])	One student record
`Node`	data (Student), left, right (Node pointers)	One tree node
`StudentBST`	root (Node *), comparisons (long)	Tree handle; functions insert, search, delete_node, traversals, height

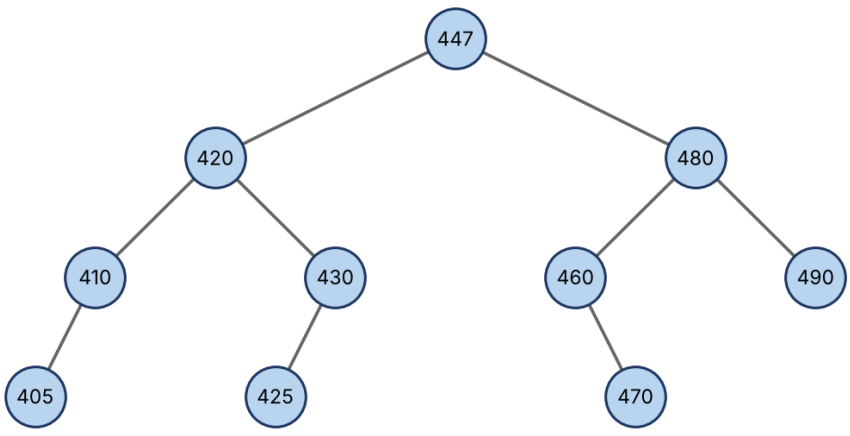


Figure 1: BST built by the 10 sample records (IDs shown as last 3 digits; actual ID = 2556xxx). Height = 4.

4. Algorithms

Insertion: compare the new key with the current node; go left if smaller, right if larger, and attach the new node at the first empty (NULL) position. If the key already exists, reject it (unique key).

Search: start at the root, compare, and move left or right until the key is found or a NULL link is reached.

Deletion has three cases:

- **Case 1 - Leaf node:** simply remove it (parent link becomes NULL) and free the node.
- **Case 2 - One child:** replace the node by its only child and free the node.
- **Case 3 - Two children:** find the *inorder successor* (smallest node of the right subtree), copy its data into the node, then delete the successor from the right subtree.

pseudocode

```
delete(node, key):
  if node is NULL: return NULL
  if key < node.key: node.left  = delete(node.left, key)
  else if key > node.key: node.right = delete(node.right, key)
  else:
      if no left and no right: free(node); return NULL        /* leaf */
      if no left:  child = node.right; free(node); return child  /* one child */
      if no right: child = node.left;  free(node); return child  /* one child */
      succ = minimum(node.right)                              /* two children */
      node.data  = succ.data
      node.right = delete(node.right, succ.key)
  return node
```

Traversals: Inorder = Left, Node, Right (sorted output); Preorder = Node, Left, Right (useful to copy a tree); Postorder = Left, Right, Node (useful to delete a tree - the program's `free_nodes()` uses this order). Each takes O(n) time. The three traversals share one pattern; each collects pointers to the records into an array, which the menu then prints.

5. Sample Input and Output

5.1 Insertion of 10 sample records (menu option 2) and duplicate rejection (option 1)

Input: Load 10 sample records, then insert ID 2556447 again.

```
10 sample records loaded.
Insert ID 2556447 again:
  [!] Duplicate ID 2556447: record rejected.
```

5.2 Search (menu option 3)

```
Search 2556430 -> FOUND -> ID: 2556430 | Name: Karan | Dept: CSE
Search 999 -> Record with ID 999 NOT found.
```

5.3 Traversals (menu options 5, 6, 7)

```
INORDER (sorted) : 2556405, 2556410, 2556420, 2556425, 2556430, 2556447, 2556460, 2556470, 2556480, 2556490
PREORDER : 2556447, 2556420, 2556410, 2556405, 2556430, 2556425, 2556480, 2556460, 2556470, 2556490
POSTORDER : 2556405, 2556410, 2556425, 2556430, 2556420, 2556470, 2556460, 2556490, 2556480, 2556447
```

5.4 Deletion - all three cases (menu option 4, in this order)

```
Delete 2556405 (Leaf node):
  (Case 1: leaf node deleted)
  Inorder IDs now: [2556410, 2556420, 2556425, 2556430, 2556447, 2556460, 2556470, 2556480, 2556490]
Delete 2556460 (One-child node):
  (Case 2: node with one (right) child deleted)
  Inorder IDs now: [2556410, 2556420, 2556425, 2556430, 2556447, 2556470, 2556480, 2556490]
Delete 2556420 (Two-children node):
  (Case 3: two children; replaced by inorder successor 2556425)
  Inorder IDs now: [2556410, 2556425, 2556430, 2556447, 2556470, 2556480, 2556490]
```

6. Balanced vs Sorted Insertion - Comparison

The same 15 keys were inserted in two orders: a balanced-looking order (50, 30, 70, 20, 40, 60, 80, ...) and ascending sorted order. The number of key comparisons during insertion, the comparisons for searching every key once, and the final tree height were counted (menu option 9).

```
TERMINAL - bash

COMPARISON: Balanced vs Sorted insertion (15 keys)
-----------------------------------------------------------------
Insertion order         Insert cmp    Search cmp    Height
Balanced-looking order  34            49            4
Sorted order            105           120           15
Balanced tree height ~ log2(n); sorted input -> skewed tree, height = n.
```

Insertion order	Insert comparisons	Search comparisons	Tree height	Shape
Balanced-looking	34	49	4	Bushy, about log2(n)
Sorted (ascending)	105	120	15	Skewed - behaves like a linked list

Result: sorted insertion needed about 3.1 times more comparisons to insert and 2.4 times more to search, because the tree degenerated into a chain of height 15 instead of 4.

7. Time and Space Complexity

Operation	Average case (balanced shape)	Worst case (skewed tree)
Insert	O(log n)	O(n)
Search	O(log n)	O(n)
Delete	O(log n)	O(n)
Inorder / Preorder / Postorder	O(n)	O(n)
Height	O(n) - visits every node	O(n)
Space (storing n records)	O(n)	O(n)
Extra space for recursion	O(log n) stack depth	O(n) stack depth

Why a regular BST can degrade to O(n): every operation walks one root-to-leaf path, so its cost is proportional to the tree height h. A BST does not rebalance itself. If keys arrive in sorted (or reverse-sorted) order, each new key becomes the right (or left) child of the previous one, producing a skewed tree of height n. Searching then needs up to n comparisons - the same as a linked list. In C the recursive insert and delete also use one stack frame per level, so a skewed tree of very large n could overflow the call stack.

How an AVL tree fixes this: an AVL tree is a self-balancing BST that stores a balance factor (height of left subtree minus height of right subtree) in every node and keeps it within -1, 0 or +1. After each insertion or deletion, if a node becomes unbalanced, the tree performs rotations (LL, RR, LR, RL) to restore balance. This keeps the height at O(log n), so search, insert and delete are guaranteed O(log n) even for sorted input.

8. Test Cases (Expected vs Actual)

ID	Test case	Expected output	Actual output	Result
TC1	Delete leaf node 20 (tree: 50,30,70,20,40,60,80,35)	Inorder [30, 35, 40, 50, 60, 70, 80]	Inorder [30, 35, 40, 50, 60, 70, 80]	PASS
TC2	Delete node 40 with one child (35) (same tree)	Inorder [20, 30, 35, 50, 60, 70, 80]	Inorder [20, 30, 35, 50, 60, 70, 80]	PASS
TC3	Delete node 50 (root) with two children (same tree)	Inorder [20, 30, 35, 40, 60, 70, 80]; new root 60	Inorder [20, 30, 35, 40, 60, 70, 80]; new root 60	PASS
TC4	Insert duplicate ID 30	Rejected with message	Rejected (returned 0)	PASS
TC5	Search non-existing ID 99	Not found (NULL)	Not found (NULL)	PASS
TC6	Inorder after inserting 10 sample records (option 5)	IDs in ascending order	Ascending: True	PASS

Raw program output of menu option 10:

```
TERMINAL - bash
$ ./student_bst

============================================================
     STUDENT RECORD SEARCH SYSTEM (Binary Search Tree)
============================================================
1. Insert student record
2. Load 10 sample records
3. Search by enrolment ID
4. Delete by enrolment ID
5. Inorder traversal (sorted order)
6. Preorder traversal
7. Postorder traversal
8. Tree height
9. Compare balanced vs sorted insertion
10. Run test cases
0. Exit
Enter choice: 10

============================================================
TEST CASES
============================================================

TC1 Delete LEAF node (20)
  (Case 1: leaf node deleted)
  Expected inorder: [30, 35, 40, 50, 60, 70, 80]
  Actual   inorder: [30, 35, 40, 50, 60, 70, 80]  -> PASS

TC2 Delete node with ONE child (40, child 35)
  (Case 2: node with one (left) child deleted)
  Expected inorder: [20, 30, 35, 50, 60, 70, 80]
  Actual   inorder: [20, 30, 35, 50, 60, 70, 80]  -> PASS

TC3 Delete node with TWO children (50 = root)
  (Case 3: two children; replaced by inorder successor 60)
  Expected inorder: [20, 30, 35, 40, 60, 70, 80]   (new root = 60)
  Actual   inorder: [20, 30, 35, 40, 60, 70, 80]  (new root = 60)  -> PASS

TC4 Duplicate insertion (30)
  [!] Duplicate ID 30: record rejected.
  Expected: rejected (0) | Actual: 0 -> PASS

TC5 Search unsuccessful (99)
  Expected: NULL | Actual: NULL -> PASS

============================================================
     STUDENT RECORD SEARCH SYSTEM (Binary Search Tree)
============================================================
   [menu options 1-10 and 0 as shown above]
Enter choice: 0
Thank you!
```

9. Source Code and Program Output (VS Code)

The code is written in student_bst.c, compiled with GCC (`gcc -Wall -Wextra -o student_bst student_bst.c`, no warnings) and executed from the VS Code terminal. Each block below shows the source code or the actual output of the menu option mentioned.

9.1 Source code - Student, Node, helpers, search and insert

File: student_bst.c (lines 12-65)

student_bst.c - Visual Studio Code

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct Student {
    int  enrol_id;          /* BST key (unique) */
    char name[32];
    char dept[16];
} Student;

typedef struct Node {
    Student data;
    struct Node *left;
    struct Node *right;
} Node;

typedef struct {
    Node *root;
    long  comparisons;      /* counts key comparisons (for analysis) */
} StudentBST;

/* ---------- Helpers ---------- */
static Student make_student(int id, const char *name, const char *dept) {
    Student s;
    s.enrol_id = id;
    strncpy(s.name, name, sizeof(s.name) - 1);  s.name[sizeof(s.name) - 1] = '\0';
    strncpy(s.dept, dept, sizeof(s.dept) - 1);  s.dept[sizeof(s.dept) - 1] = '\0';
    return s;
}

static void print_student(const Student *s) {
    printf("  ID: %d | Name: %-10s | Dept: %s\n", s->enrol_id, s->name, s->dept);
}

static void tree_init(StudentBST *t) { t->root = NULL; t->comparisons = 0; }

static void free_nodes(Node *n) {
    if (n) { free_nodes(n->left); free_nodes(n->right); free(n); }
}

static int count_nodes(Node *n) {
    return n ? 1 + count_nodes(n->left) + count_nodes(n->right) : 0;
}

/* ---------- Search ---------- */
static Node *search(StudentBST *t, int id, int count) {
    Node *cur = t->root;
    while (cur) {
        if (count) t->comparisons++;
        if (id == cur->data.enrol_id) return cur;
        cur = (id < cur->data.enrol_id) ? cur->left : cur->right;
    }
    return NULL;
}
```

File: student_bst.c (lines 67-91)

student_bst.c - Visual Studio Code

```c
/* ---------- Insert ---------- */
static Node *insert_node(StudentBST *t, Node *node, Student s) {
    if (node == NULL) {
        Node *n = (Node *)malloc(sizeof(Node));
        if (!n) { fprintf(stderr, "Out of memory\n"); exit(1); }
        n->data = s; n->left = n->right = NULL;
        return n;
    }
    t->comparisons++;
    if (s.enrol_id < node->data.enrol_id)
        node->left = insert_node(t, node->left, s);
    else
        node->right = insert_node(t, node->right, s);
    return node;
}

/* Returns 0 for a duplicate enrolment ID, 1 on success. */
static int insert(StudentBST *t, Student s) {
    if (search(t, s.enrol_id, 0)) {
        printf("  [!] Duplicate ID %d: record rejected.\n", s.enrol_id);
        return 0;
    }
    t->root = insert_node(t, t->root, s);
    return 1;
}
```

9.2 Source code - delete, traversals and height

File: student_bst.c (lines 93-140)

student_bst.c - Visual Studio Code

```c
/* ---------- Delete ---------- */
static Node *delete_node(Node *node, int key, int quiet) {
    if (node == NULL) return NULL;
    if (key < node->data.enrol_id) {
        node->left = delete_node(node->left, key, quiet);
    } else if (key > node->data.enrol_id) {
        node->right = delete_node(node->right, key, quiet);
    } else {
        /* Case 1: leaf node */
        if (node->left == NULL && node->right == NULL) {
            if (!quiet) printf("  (Case 1: leaf node deleted)\n");
            free(node);
            return NULL;
        }
        /* Case 2: one child */
        if (node->left == NULL) {
            Node *child = node->right;
            if (!quiet) printf("  (Case 2: node with one (right) child deleted)\n");
            free(node);
            return child;
        }
        if (node->right == NULL) {
            Node *child = node->left;
            if (!quiet) printf("  (Case 2: node with one (left) child deleted)\n");
            free(node);
            return child;
        }
        /* Case 3: two children -> replace with inorder successor */
        Node *succ = node->right;
        while (succ->left) succ = succ->left;
        if (!quiet)
            printf("  (Case 3: two children; replaced by inorder successor %d)\n",
                   succ->data.enrol_id);
        node->data = succ->data;
        node->right = delete_node(node->right, succ->data.enrol_id, 1);
    }
    return node;
}

/* Returns 1 if deleted, 0 if the ID was not found. */
static int delete_id(StudentBST *t, int id) {
    if (!search(t, id, 0)) {
```

student_bst.c - Visual Studio Code (continued)

```
        printf("  [!] ID %d not found. Nothing deleted.\n", id);
        return 0;
    }
    t->root = delete_node(t->root, id, 0);
    return 1;
}
```

File: student_bst.c (lines 142-178)

student_bst.c - Visual Studio Code

```
/* ---------- Traversals (collect pointers into out[]) ---------- */
static void inorder(Node *n, Student **out, int *k) {
    if (n) { inorder(n->left, out, k); out[(*k)++] = &n->data; inorder(n->right, out, k); }
}
static void preorder(Node *n, Student **out, int *k) {
    if (n) { out[(*k)++] = &n->data; preorder(n->left, out, k); preorder(n->right, out, k); }
}
static void postorder(Node *n, Student **out, int *k) {
    if (n) { postorder(n->left, out, k); postorder(n->right, out, k); out[(*k)++] = &n->data; }
}

static int height(Node *n) {
    if (n == NULL) return 0;
    int l = height(n->left), r = height(n->right);
    return 1 + (l > r ? l : r);
}

typedef void (*TraverseFn)(Node *, Student **, int *);

static void show_traversal(StudentBST *t, const char *title, TraverseFn fn) {
    int n = count_nodes(t->root), k = 0;
    printf("%s\n", title);
    if (n == 0) { printf("  (tree is empty)\n"); return; }
    Student **out = (Student **)malloc(sizeof(Student *) * n);
    fn(t->root, out, &k);
    for (int i = 0; i < k; i++) print_student(out[i]);
    free(out);
}

static void keys_of(StudentBST *t, int *keys, int *n) {
    int cnt = count_nodes(t->root), k = 0;
    Student **out = (Student **)malloc(sizeof(Student *) * (cnt ? cnt : 1));
    inorder(t->root, out, &k);
    for (int i = 0; i < k; i++) keys[i] = out[i]->enrol_id;
    *n = k;
    free(out);
}
```

9.3 Output - compile, insertion of 10 records and duplicate rejection (options 2 and 1)

```
TERMINAL - bash

$ gcc -Wall -Wextra -o student_bst student_bst.c
$ ./student_bst

============================================================
     STUDENT RECORD SEARCH SYSTEM (Binary Search Tree)
============================================================
1. Insert student record
2. Load 10 sample records
3. Search by enrolment ID
4. Delete by enrolment ID
5. Inorder traversal (sorted order)
6. Preorder traversal
7. Postorder traversal
8. Tree height
9. Compare balanced vs sorted insertion
10. Run test cases
0. Exit
Enter choice: 2
  10 sample records loaded.

============================================================
     STUDENT RECORD SEARCH SYSTEM (Binary Search Tree)
============================================================
   [menu options 1-10 and 0 as shown above]
Enter choice: 1
Enrolment ID: 2556447
Name: Dhruv
Department: CSE
  [!] Duplicate ID 2556447: record rejected.

============================================================
     STUDENT RECORD SEARCH SYSTEM (Binary Search Tree)
============================================================
   [menu options 1-10 and 0 as shown above]
Enter choice: 0
Thank you!
```

9.4 Output - successful and unsuccessful search (option 3)

```
TERMINAL - bash

$ ./student_bst

============================================================
     STUDENT RECORD SEARCH SYSTEM (Binary Search Tree)
============================================================
1. Insert student record
2. Load 10 sample records
3. Search by enrolment ID
4. Delete by enrolment ID
5. Inorder traversal (sorted order)
6. Preorder traversal
7. Postorder traversal
8. Tree height
9. Compare balanced vs sorted insertion
10. Run test cases
0. Exit
Enter choice: 2
  10 sample records loaded.

============================================================
     STUDENT RECORD SEARCH SYSTEM (Binary Search Tree)
============================================================
   [menu options 1-10 and 0 as shown above]
Enter choice: 3
Enrolment ID to search: 2556430
  FOUND -> ID: 2556430 | Name: Karan      | Dept: CSE

============================================================
     STUDENT RECORD SEARCH SYSTEM (Binary Search Tree)
============================================================
   [menu options 1-10 and 0 as shown above]
Enter choice: 3
Enrolment ID to search: 999
  Record with ID 999 NOT found.

============================================================
     STUDENT RECORD SEARCH SYSTEM (Binary Search Tree)
============================================================
   [menu options 1-10 and 0 as shown above]
Enter choice: 0
Thank you!
```

9.5 Output - inorder, preorder and postorder (options 5, 6, 7)

```
TERMINAL - bash

$ ./student_bst

============================================================
     STUDENT RECORD SEARCH SYSTEM (Binary Search Tree)
============================================================
1. Insert student record
2. Load 10 sample records
3. Search by enrolment ID
4. Delete by enrolment ID
5. Inorder traversal (sorted order)
6. Preorder traversal
7. Postorder traversal
8. Tree height
9. Compare balanced vs sorted insertion
10. Run test cases
0. Exit
Enter choice: 2
  10 sample records loaded.

============================================================
     STUDENT RECORD SEARCH SYSTEM (Binary Search Tree)
============================================================
   [menu options 1-10 and 0 as shown above]
Enter choice: 5
INORDER (sorted by ID):
  ID: 2556405 | Name: Pooja      | Dept: CSE
  ID: 2556410 | Name: Riya       | Dept: ECE
  ID: 2556420 | Name: Aman       | Dept: CSE
  ID: 2556425 | Name: Vikas      | Dept: ECE
  ID: 2556430 | Name: Karan      | Dept: CSE
  ID: 2556447 | Name: Dhruv      | Dept: CSE
  ID: 2556460 | Name: Neha       | Dept: IT
  ID: 2556470 | Name: Anjali     | Dept: CSE
  ID: 2556480 | Name: Simran     | Dept: IT
  ID: 2556490 | Name: Rohan      | Dept: ME

============================================================
     STUDENT RECORD SEARCH SYSTEM (Binary Search Tree)
============================================================
   [menu options 1-10 and 0 as shown above]
Enter choice: 6
PREORDER:
  ID: 2556447 | Name: Dhruv      | Dept: CSE
  ID: 2556420 | Name: Aman       | Dept: CSE
  ID: 2556410 | Name: Riya       | Dept: ECE
  ID: 2556405 | Name: Pooja      | Dept: CSE
  ID: 2556430 | Name: Karan      | Dept: CSE
  ID: 2556425 | Name: Vikas      | Dept: ECE
  ID: 2556480 | Name: Simran     | Dept: IT
  ID: 2556460 | Name: Neha       | Dept: IT
  ID: 2556470 | Name: Anjali     | Dept: CSE
  ID: 2556490 | Name: Rohan      | Dept: ME

============================================================
     STUDENT RECORD SEARCH SYSTEM (Binary Search Tree)
============================================================
   [menu options 1-10 and 0 as shown above]
Enter choice: 7
POSTORDER:
  ID: 2556405 | Name: Pooja      | Dept: CSE
  ID: 2556410 | Name: Riya       | Dept: ECE
  ID: 2556425 | Name: Vikas      | Dept: ECE
  ID: 2556430 | Name: Karan      | Dept: CSE
  ID: 2556420 | Name: Aman       | Dept: CSE
  ID: 2556470 | Name: Anjali     | Dept: CSE
  ID: 2556460 | Name: Neha       | Dept: IT
  ID: 2556490 | Name: Rohan      | Dept: ME
  ID: 2556480 | Name: Simran     | Dept: IT
  ID: 2556447 | Name: Dhruv      | Dept: CSE
```

TERMINAL - bash (continued)

```
============================================================
     STUDENT RECORD SEARCH SYSTEM (Binary Search Tree)
============================================================
   [menu options 1-10 and 0 as shown above]
Enter choice: 0
Thank you!
```

9.6 Output - deletion: leaf, one child, two children (option 4), then inorder

```
TERMINAL - bash

$ ./student_bst

============================================================
     STUDENT RECORD SEARCH SYSTEM (Binary Search Tree)
============================================================
1. Insert student record
2. Load 10 sample records
3. Search by enrolment ID
4. Delete by enrolment ID
5. Inorder traversal (sorted order)
6. Preorder traversal
7. Postorder traversal
8. Tree height
9. Compare balanced vs sorted insertion
10. Run test cases
0. Exit
Enter choice: 2
  10 sample records loaded.

============================================================
     STUDENT RECORD SEARCH SYSTEM (Binary Search Tree)
============================================================
   [menu options 1-10 and 0 as shown above]
Enter choice: 4
Enrolment ID to delete: 2556405
  (Case 1: leaf node deleted)
  Record 2556405 deleted.

============================================================
     STUDENT RECORD SEARCH SYSTEM (Binary Search Tree)
============================================================
   [menu options 1-10 and 0 as shown above]
Enter choice: 4
Enrolment ID to delete: 2556460
  (Case 2: node with one (right) child deleted)
  Record 2556460 deleted.

============================================================
     STUDENT RECORD SEARCH SYSTEM (Binary Search Tree)
============================================================
   [menu options 1-10 and 0 as shown above]
Enter choice: 4
Enrolment ID to delete: 2556420
  (Case 3: two children; replaced by inorder successor 2556425)
  Record 2556420 deleted.

============================================================
     STUDENT RECORD SEARCH SYSTEM (Binary Search Tree)
============================================================
   [menu options 1-10 and 0 as shown above]
Enter choice: 5
INORDER (sorted by ID):
  ID: 2556410 | Name: Riya       | Dept: ECE
  ID: 2556425 | Name: Vikas      | Dept: ECE
  ID: 2556430 | Name: Karan      | Dept: CSE
  ID: 2556447 | Name: Dhruv      | Dept: CSE
  ID: 2556470 | Name: Anjali     | Dept: CSE
  ID: 2556480 | Name: Simran     | Dept: IT
  ID: 2556490 | Name: Rohan      | Dept: ME

============================================================
     STUDENT RECORD SEARCH SYSTEM (Binary Search Tree)
============================================================
   [menu options 1-10 and 0 as shown above]
Enter choice: 0
Thank you!
```

9.7 Output - tree height and balanced vs sorted comparison (options 8 and 9)

```
TERMINAL - bash

$ ./student_bst

============================================================
     STUDENT RECORD SEARCH SYSTEM (Binary Search Tree)
============================================================
1. Insert student record
2. Load 10 sample records
3. Search by enrolment ID
4. Delete by enrolment ID
5. Inorder traversal (sorted order)
6. Preorder traversal
7. Postorder traversal
8. Tree height
9. Compare balanced vs sorted insertion
10. Run test cases
0. Exit
Enter choice: 2
  10 sample records loaded.

============================================================
     STUDENT RECORD SEARCH SYSTEM (Binary Search Tree)
============================================================
   [menu options 1-10 and 0 as shown above]
Enter choice: 8
  Height of tree = 4

============================================================
     STUDENT RECORD SEARCH SYSTEM (Binary Search Tree)
============================================================
   [menu options 1-10 and 0 as shown above]
Enter choice: 9

COMPARISON: Balanced vs Sorted insertion (15 keys)
------------------------------------------------------------------
Insertion order         Insert cmp    Search cmp    Height
Balanced-looking order  34            49            4
Sorted order            105           120           15
Balanced tree height ~ log2(n); sorted input -> skewed tree, height = n.

============================================================
     STUDENT RECORD SEARCH SYSTEM (Binary Search Tree)
============================================================
   [menu options 1-10 and 0 as shown above]
Enter choice: 0
Thank you!
```

9.8 Output - test cases (option 10)

All 5 test cases PASS (full output shown in Section 8).

Note: in the terminal transcripts above, the menu is printed in full once; later repetitions of the identical menu are abbreviated to save space.

10. Conclusion

The Student Record Search System was implemented in C using a Binary Search Tree with dynamic memory (`malloc`/`free`), insertion (duplicate IDs rejected), search, three-case deletion, and all three traversals. Inorder traversal produces records sorted by enrolment ID. The experiment showed that sorted insertion produces a skewed tree with height equal to n and many more comparisons than a balanced insertion order, which is why self-balancing trees such as AVL trees are used in practice. All test cases passed. Future scope: AVL / Red-Black tree implementation, saving records to a file, and search by name.

GitHub Repository Link: _____ (paste your link here)